Blynk IoT User Manual

Step-by-Step Hardware Integration for Outputs (LEDs) and Sensor Inputs

This user manual provides systematic, step-by-step instructions to configure the Blynk IoT cloud platform for controlling physical output devices (like LEDs or Relays) and monitoring remote data feeds using microcontrollers (ESP32 / ESP8266).

Section 1: The Cloud Architecture Blueprint

Blynk IoT functions by separating hardware interactions into independent streams called **Datastreams**. Instead of sending hardware execution commands straight to GPIO pins, you configure virtual software links called **Virtual Pins**. This architecture ensures software stability, asynchronous data exchanges, and uniform controls regardless of whether you are deploying an ESP32 or an ESP8266.

Section 2: Configuring the Blynk Web Console

Before deploying any source code to your microchip, the cloud framework profile must be established within the Blynk Developer Console portal.

Step 2.1: Create a Device Template

Log into your cloud console account. Navigate to the **Developer Zone** (wrench icon) from the sidebar menu, select **Templates**, and click **+ New Template**. Assign a system name, configure the hardware type to match your chip target, and set the connection type to **WiFi**.

Step 2.2: Establish Virtual Datastreams

Inside the newly created template, choose the **Datastreams** tab. You must create individual virtual pathways to partition output controls from incoming sensor values. Configure your channels strictly using the following parameters:

Virtual Pin	Name / Destination Role	Data Type	Min Limit	Max Limit (ESP8266 / ESP32)
V1	LED Switch Controller	Integer	0	1 (Binary State Switch)
V2	System Diagnostic Uptime	Integer	0	Unlimited ($t > 0$)
V3	Analog Sensor Data Feed	Integer	0	1023 (ESP8266) / 4095 (ESP32)

Step 2.3: **Acquire Device Credentials**

Navigate to the search layout dashboard, select **+ New Device**, and link it directly to your template. Click on the **Device Info** metadata sub-panel. On the right side layout, copy the generated **Firmware Configuration** text block containing the matching IDs and Authorization Tokens.

Section 3: Unified Cross-Platform Source Code

The system source layout handles asynchronous non-blocking background loops. Choose the exact matching distribution structure based on your specific target microcontroller architecture:

Option A: Deployment Profile Code for ESP32 Architecture

```
#define BLYNK_TEMPLATE_ID "TMPL3FhO-xtB_"
#define BLYNK_TEMPLATE_NAME "TEST"
#define BLYNK_AUTH_TOKEN "AuS4lfNFfx4lSFmTokvyEPkz0Jg_rQXC"

#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Airtel_kaml_0228";
char pass[] = "Passward@078078";

const int ledPin = 2;        // Built-in LED on GPIO 2
const int sensorPin = A0;     // Corresponds to VP / GPIO 36

BlynkTimer timer;

void myTimerEvent() {
    Blynk.virtualWrite(V2, millis() / 1000);
    int rawAnalogValue = analogRead(sensorPin);
    Blynk.virtualWrite(V3, rawAnalogValue);
    Serial.print("ESP32 Sensor Level (A0): ");
    Serial.println(rawAnalogValue);
}

BLYNK_WRITE(V1) {
    int state = param.asInt();
    digitalWrite(ledPin, state);
}

void setup() {
    Serial.begin(115200);
    pinMode(ledPin, OUTPUT);
    pinMode(sensorPin, INPUT);
    Blynk.begin(auth, ssid, pass);
    timer.setInterval(1000L, myTimerEvent);
}

void loop() {
    Blynk.run();
    timer.run();
}
```

> **Hardware Focus (ESP32):** The internal Analog-to-Digital Converter operates a 12-bit register resolution mapping mathematical sensor boundaries dynamically from $0 \le ext{Value} \le 4095$.

Option B: Deployment Profile Code for ESP8266 (NodeMCU) Architecture

```
#define BLYNK_TEMPLATE_ID "TMPL3FhO-xtB_"
#define BLYNK_TEMPLATE_NAME "TEST"
#define BLYNK_AUTH_TOKEN "AuS4lfNFfx4lSFmTokvyEPkz0Jg_rQXC"

#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Airtel_kaml_0228";
char pass[] = "Passward@078078";

const int ledPin = D4;       // Built-in Active-LOW LED on Pin D4
const int sensorPin = A0;     // Dedicated Single Analog Input A0

BlynkTimer timer;

void myTimerEvent() {
    Blynk.virtualWrite(V2, millis() / 1000);
    int rawAnalogValue = analogRead(sensorPin);
    Blynk.virtualWrite(V3, rawAnalogValue);
    Serial.print("ESP8266 Sensor Level (A0): ");
    Serial.println(rawAnalogValue);
}

BLYNK_WRITE(V1) {
    int state = param.asInt();
    digitalWrite(ledPin, state);
}

void setup() {
    Serial.begin(115200);
    pinMode(ledPin, OUTPUT);
    pinMode(sensorPin, INPUT);
    Blynk.begin(auth, ssid, pass);
    timer.setInterval(1000L, myTimerEvent);
}

void loop() {
    Blynk.run();
    timer.run();
}
```

> **Hardware Focus (ESP8266):** The analog input subsystem operates an explicit 10-bit converter array scaling calculations strictly between $0 \le ext{Value} \le 1023$.

Section 4: Interface Mobile Layout Integration

With hardware profiles listening over network links, execute your mobile configuration layout to link active visual display widgets to your backend code variables.

Step 4.1: Setup Remote Switching (Output Control)

Open your mobile Blynk App and select your active target template workspace. From the utility widget tray layout, drag a **Button Widget** into your main panel context. Enter the setting parameters, assign the datastream value channel precisely to **Virtual Pin V1**, and convert its functional behavior mode from *Push* to **Switch**. This safely interfaces hardware state changes.

Step 4.2: Setup Analog Sensor Displays (Input Monitoring)

Reopen your widget layout canvas panel tray. Drag out an interface **Gauge Widget** or a **Value Display Widget**. Set its source datastream link mapping directly to **Virtual Pin V3**. Define the data update processing command loop as **PUSH** to capture instant transmission sweeps whenever your software triggers a `Blynk.virtualWrite()` payload event.

Section 5: Crucial Maintenance Rules for Stable Code

- **Strict Prohibition of Loop Delays:** Never embed hardware `delay()` routines anywhere within the main execution layout loop. Halting the runtime environment breaks the structural keep-alive ping intervals, causing instant connection drops.
- **Enforce Timed Interruptions:** Run any code meant to read sensors or execute recurring checks inside an explicit timed interrupt loop handled via `BlynkTimer` objects.
- **Pin Limitations:** Ensure that any analog inputs connected to your ESP32 avoid ADC2 lines while Wi-Fi tasks run. Stick to ADC1 channels (such as Pin VP / GPIO 36) to prevent data drops.